

Full-Stack

Flexing

with

Ruby on Rails





Ruby on Rails

is back at

CodeMash!

***A Slight History
Detour...***



Jon Hunt
jdhunt



JohnCrider
JohnCrider

srallya
from Twitter (13 min ago)
#codemash Music by 'Two Steps from Hell' is great to get the adrenaline pumping to do some coding.

SirCaliban
from Twitter (just now)
RT @chrismarinos: Video games don't make you happy? Speak for yourself! (sent from my smartphone)
#CodeMash

nathjwartz
from Twitter (just now)
You know what, @jonsieet makes me happy #CodeMash

imaung
from Twitter (just now)
#robots makes me feel happy #codemash

coridrew
from Twitter (13 min ago)
Feeling guilty for thinking how happy coding makes me. Was that this speaker's intention? YES- Technology DOES make me VERY HAPPY!
#codemash

Jimmy @yrd
from Twitter (just now)
I recorded all the Ignite speeches if anyone wants them #codemash

dsbo
from Twitter (13 min ago)
happy when I sh



The Golden Era:

2008-2013

Jim Weirich

1956 - 2014



Charles Oliver Nutter



Nick Quaranto,

Joe O'Brien,

Matt Yoho,

Jeff Casimir,

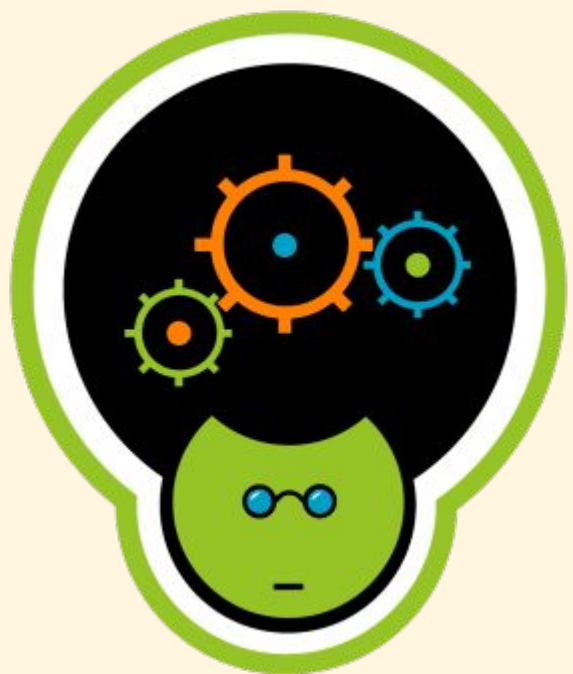
Justin Searls

and many more!

"React on Rails"

by

Josh Graber



2017





*Rob
Stevenson*

Columbus Ruby Brigade



Previous CodeMash talks:

- Ruby on Rails for Java Developers (2007)
- JRuby: Wonder Twin Powers, Activate! (2008)
- FlexMock: We Don't Need No Stinkin' Objects! (2008)

Be My RubyFriend!



<https://rubyfriends.app/profiles/BMBY>

Ruby On Rails

Version 8.1





Ruby On Rails

Version 8.1

Solid Cable

Solid Cache

Solid Queue



Ruby On Rails

Version 8.1

*Thruster -
HTTP/2 Proxy*



Ruby On Rails

Version 8.1

Kamal 2.0 -

Kamal Proxy



Ruby On Rails

Version 8.1

Propshaft



Ruby on Rails

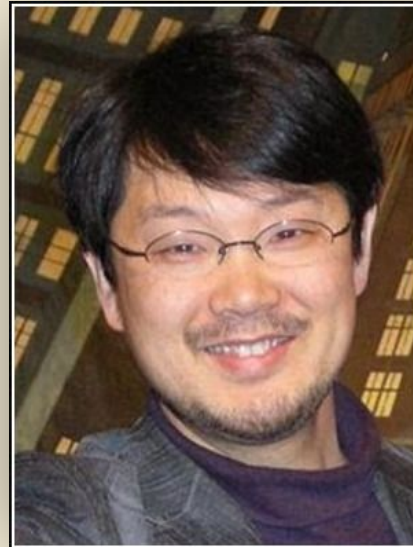
Secret

Weapon

Ruby!



Yukihiro "Matz" Matsumoto



I believe that the purpose of life is, at least in part, to be happy. Based on this belief, Ruby is designed to make programming not only easy but also fun. It allows you to concentrate on the creative side of programming, with less stress.

— Yukihiro Matsumoto —

AZ QUOTES

“I hope to see Ruby help every programmer in the world to be productive, and to enjoy programming, and to be happy. That is the primary purpose of Ruby language. Ruby is designed to make programming not only easy but also fun, allowing users to concentrate on the creative side of programming with less stress. I designed Ruby for humans, not for machines, with productivity as one of its primary goals. The language is intended to be simple in appearance but complex inside, much like the human body.”



MINASWAN

Ruby 4.0



Even primitives have methods – Everything is an object in Ruby!

```
-5.abs           # => 5  
"hello".upcase  # => "HELLO"  
nil.class       # => NilClass
```

Output "I love Ruby"

```
say = "I love Ruby"  
puts say
```

Output "I *LOVE* RUBY" – Strings are mutable & expressive

```
say = say.sub("love", "*love*")  
puts say.upcase
```

Output "I *love* Ruby" five times – Blocks make iteration easy
5.times { puts say }

More fun with blocks & natural language feel
5.times { puts "Hello!" }

Natural language-like syntax – Reads like English
if user.logged_in?
 puts "Welcome back!"
end

String interpolation – So clean, so readable
name = "Alice"
puts "Hello, #{name}!"

Language changes

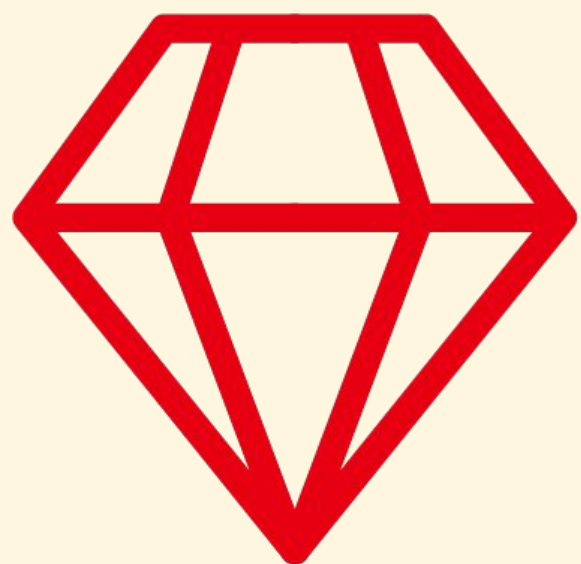
- `*nil` no longer calls `nil.to_a`, similar to how `**nil` does not call `nil.to_hash`. [\[Feature #21047\]](#)
- Logical binary operators (`||`, `&&`, `and` and `or`) at the beginning of a line continue the previous line, like fluent dot. The following code examples are equal:

```
if condition1
  && condition2
  ...
end
```

Previously:

```
if condition1 && condition2
  ...
end
```

```
if condition1 &&
  condition2
  ...
end
```



YJIT



tobi lutke



@tobi



Congratulations to our merchants on another record breaking Black Friday. LFG

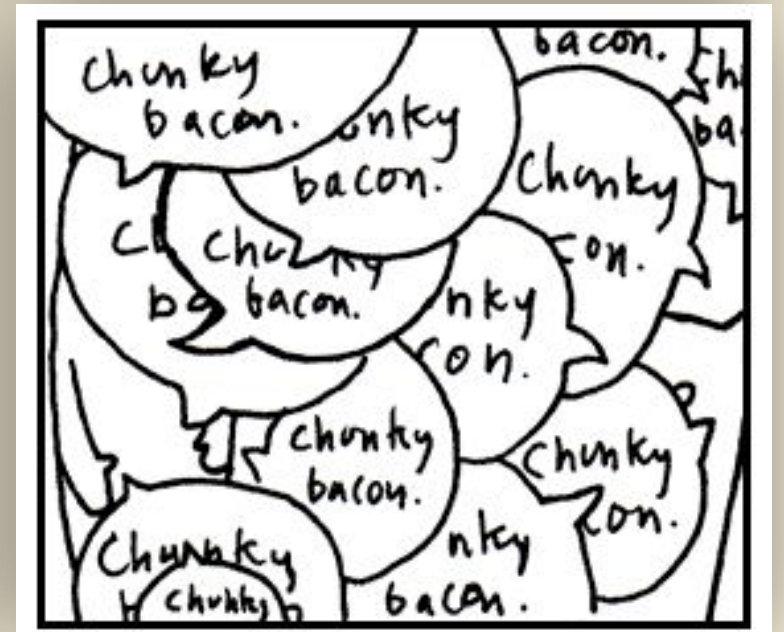
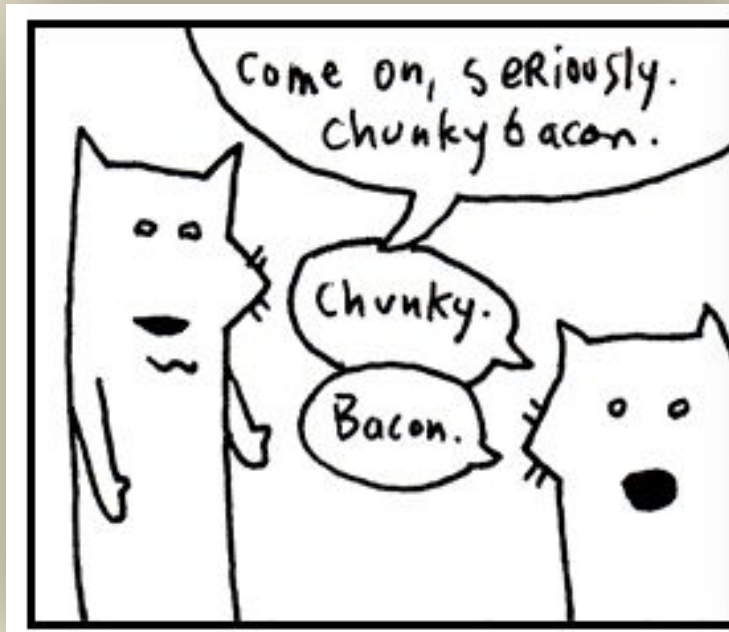
- Merchants total Black Friday sales were \$6.2 billion, up 25% YoY
- Edge peaked at 489 million requests per minute. App servers handled a peak of 117 million requests per minute, up 40% from last year
- Shopify's egress processed 237 billion requests
- Peak database queries reached over 53 million per second and writes reached over 2 million per second
- API processed over 31 million requests per minute at peak

8:57 AM · Nov 29, 2025 · **875.5K** Views



Austin's newest Ruby conference:
One day and filled with weird.

A close-up photograph of actor James Van Der Beek. He has shoulder-length, wavy brown hair and a light beard. He is wearing a dark pinstriped blazer over a white t-shirt. He is looking down, and a microphone is visible in the foreground.



Ruby!



Ruby On Rails

Version 8.1



The Rails Doctrine.

David Heinemeier Hansson

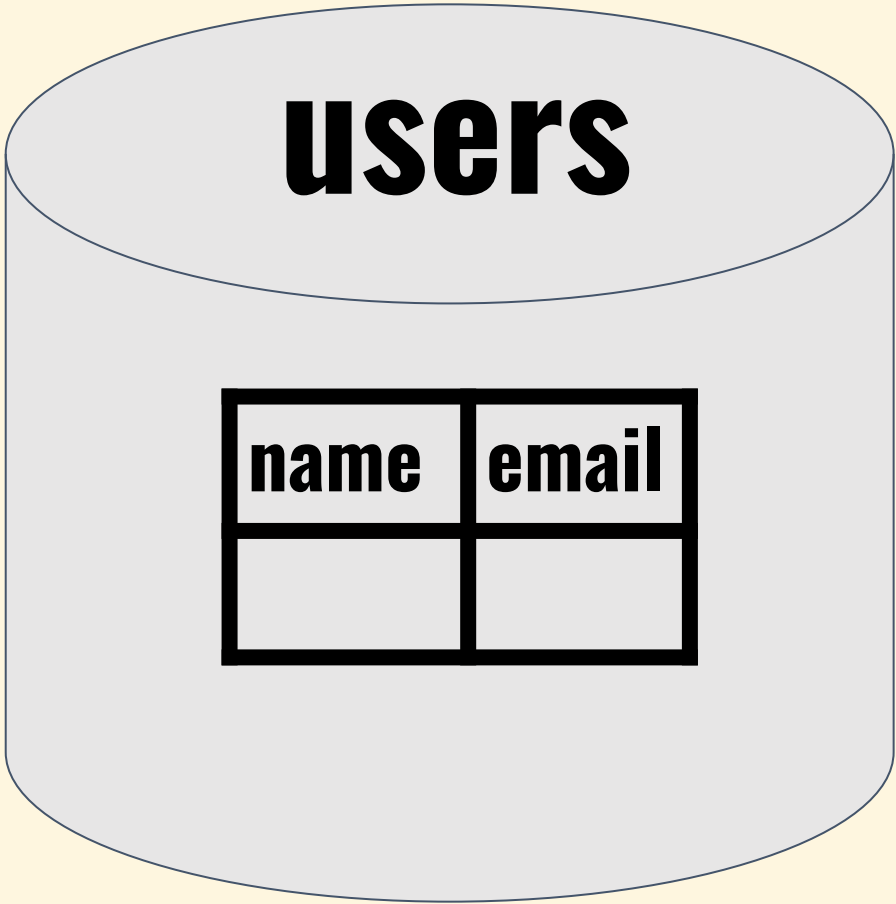
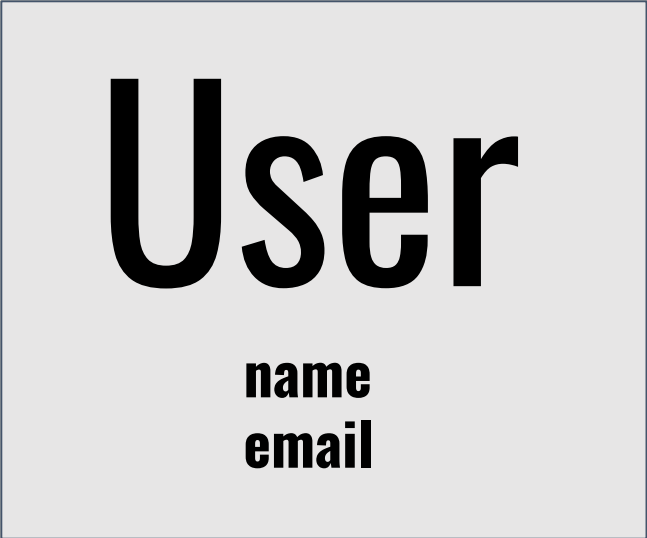
- Optimize for programmer happiness
- Convention over Configuration
- The menu is omakase
- No one paradigm
- Exalt beautiful code
- Provide sharp knives
- Value integrated systems
- Progress over stability
- Push up a big tent

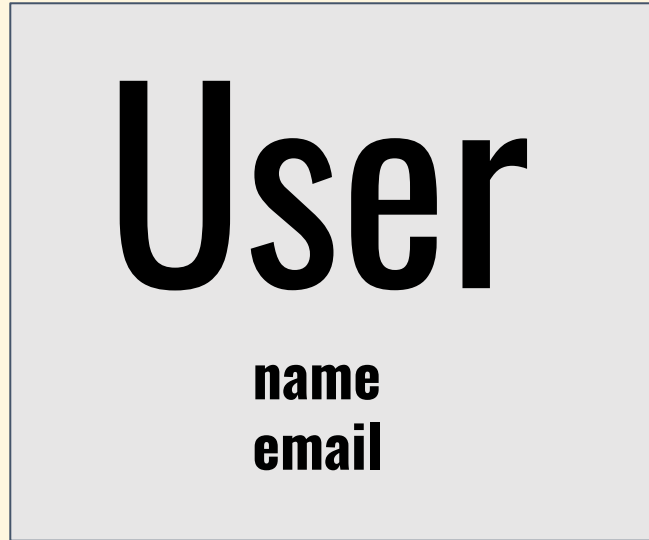
Full-Stack
Flexing



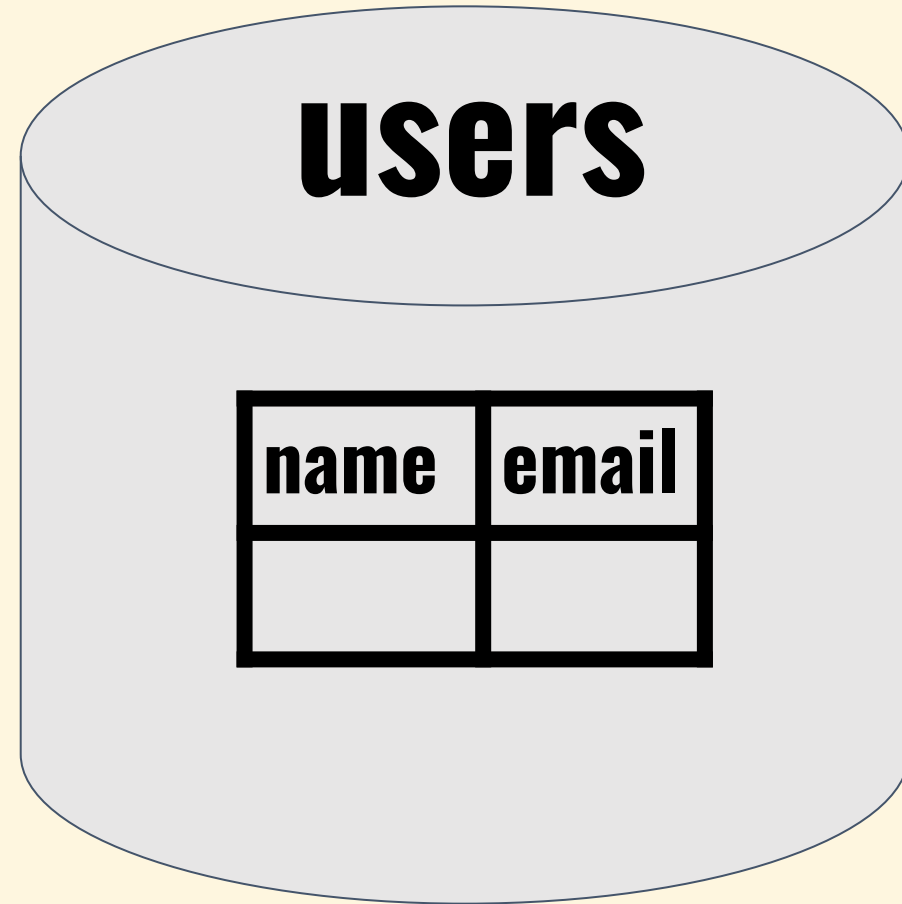
Active Record





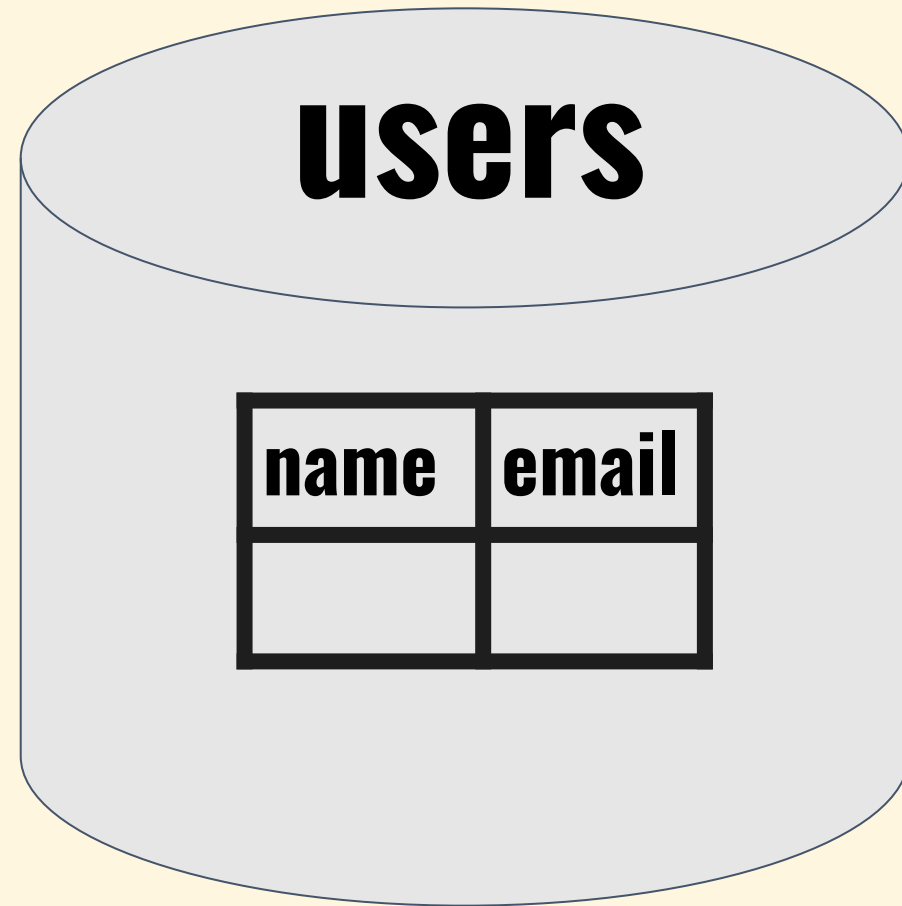


class User





class User < ApplicationRecord



Migration!

Migration!

```
$ bin/rails generate migration CreateUsers name:string email:string
```

```
invoke  active_record
```

```
create  db/migrate/20260111203045_create_users.rb
```

Migration!

```
$ bin/rails db:migrate
```

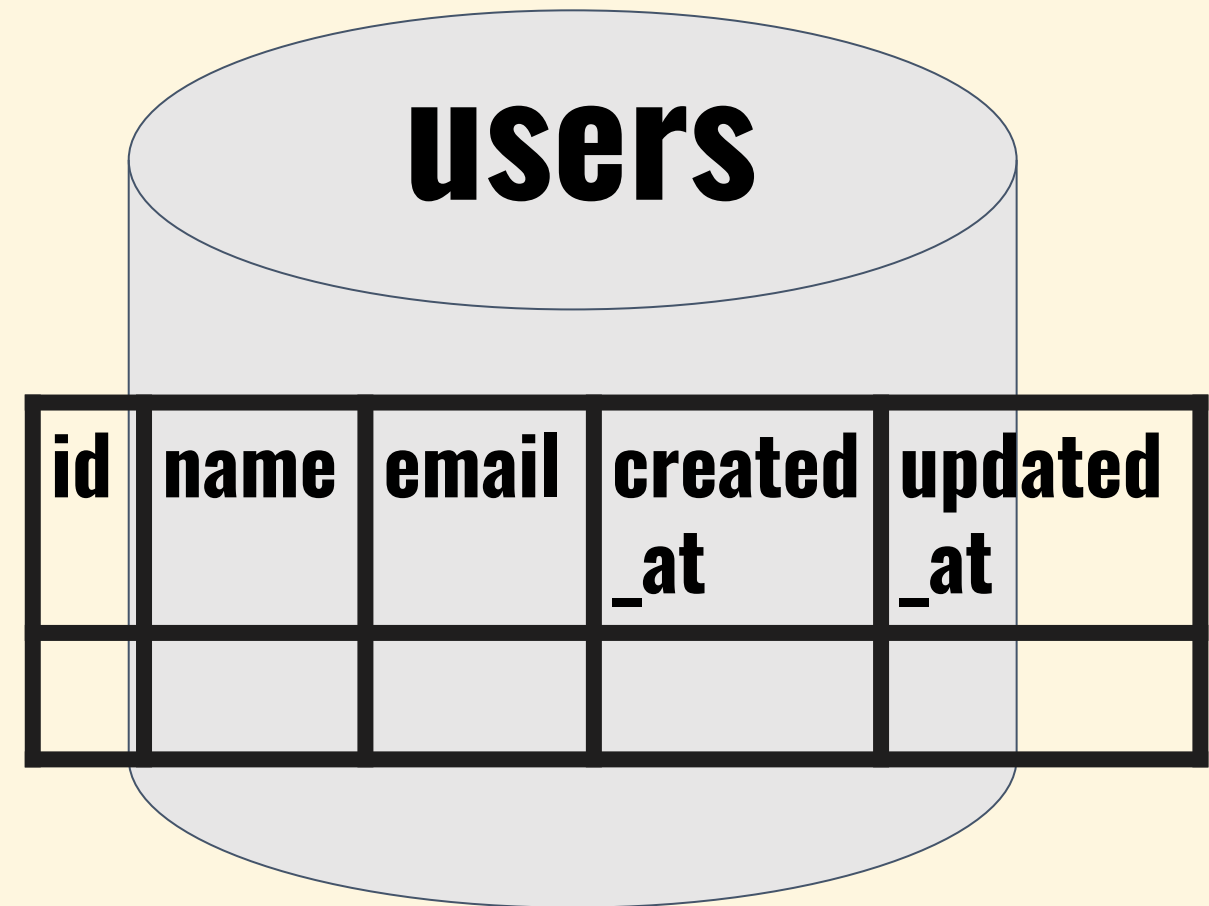
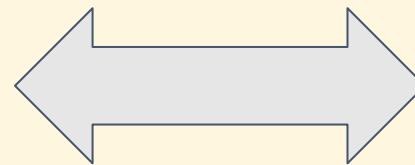
```
# db/migrate/20260111203045_create_users.rb
class CreateUsers < ActiveRecord::Migration[8.1]
  def change
    create_table :users do |t|
      t.string :name
      t.string :email
      t.timestamps
    end
  end
end
```

Migration!

```
class CreateUsers < ActiveRecord::Migration[8.1]
  def change
    create_table :users do |t|
      t.string :name, null: false
      t.string :email, limit: 255, null: false
      t.datetime "created_at", null: false
      t.datetime "updated_at", null: false
    end
  end
end
```

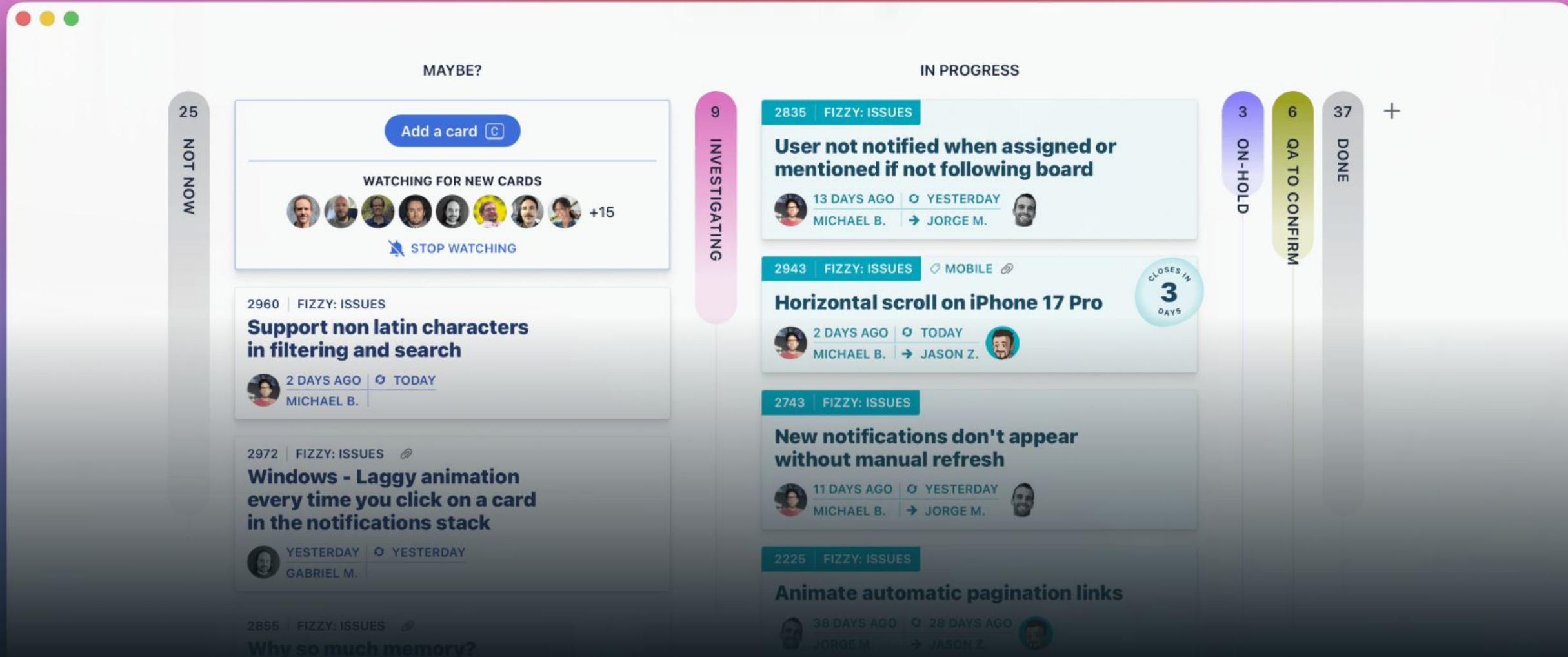
!Migration

```
$ bin/rails db:rollback
```



class User < ApplicationRecord

```
# app/models/user.rb
class User < ApplicationRecord
  validates :name, :email, presence: true
  validates :email, uniqueness: true
end
```



The screenshot displays a Kanban board with the following structure:

- Columns:**
 - NOT NOW:** 25 cards. Contains a "MAYBE?" section with an "Add a card" button and a "WATCHING FOR NEW CARDS" section with 15 user avatars and a "STOP WATCHING" button.
 - INVESTIGATING:** 9 cards.
 - IN PROGRESS:** 2835 cards. Contains three visible cards:
 - Card 1:** "User not notified when assigned or mentioned if not following board". 13 days ago, assigned to Michael B., updated yesterday by Jorge M.
 - Card 2:** "Horizontal scroll on iPhone 17 Pro". 2 days ago, assigned to Michael B., updated today by Jason Z. Includes a "CLOSES IN 3 DAYS" badge.
 - Card 3:** "New notifications don't appear without manual refresh". 11 days ago, assigned to Michael B., updated yesterday by Jorge M.
 - ON-HOLD:** 3 cards.
 - QA TO CONFIRM:** 6 cards.
 - DONE:** 37 cards.

Each card includes a title, assignee, due date, and a list of contributors.

Kanban as it should be.
Not as it has been.

```
1  ✓ class Assignment < ApplicationRecord
2      LIMIT = 100
3
4      belongs_to :account, default: -> { card.account }
5      belongs_to :card, touch: true
6
7      belongs_to :assignee, class_name: "User"
8      belongs_to :assigner, class_name: "User"
9
10     validate :within_limit, on: :create
11
12     private
13     ✓ def within_limit
14         if card.assignments.count >= LIMIT
15             errors.add(:base, "Card already has the maximum of #{LIMIT} assignees")
16         end
17     end
18 end
```

```
1  class Comment < ApplicationRecord
2    include Attachments, Eventable, Mentions, Promptable, Searchable, Storage::Tracked
3
4    belongs_to :account, default: -> { card.account }
5    belongs_to :card, touch: true
6    belongs_to :creator, class_name: "User", default: -> { Current.user }
7    has_many :reactions, -> { order(:created_at) }, dependent: :delete_all
8
9    has_rich_text :body
10
11    scope :chronologically, -> { order created_at: :asc, id: :desc }
12    scope :preloaded, -> { with_rich_text_body.includes(reactions: :reacter) }
13    scope :by_system, -> { joins(:creator).where(creator: { role: :system }) }
14    scope :by_user, -> { joins(:creator).where.not(creator: { role: :system }) }
15
16    after_create_commit :watch_card_by_creator
17
18    delegate :publicly_accessible?, :accessible_to?, :board, :watch_by, to: :card
19
```

```
# One-step create + save (most common & recommended)
user = User.create(name: "alice", email: "alice@codemash.org")

# => #<User id: 1, name: "alice", email: "alice@codemash.org",
...>
```

```
# Or instantiate first, then save (great for form handling)
user = User.new(name: "bob", email: "bob@codemash.org")
```

```
user.save          # returns true/false
user.save!         # raises ActiveRecord::RecordInvalid on
                   # failure
```

User.all	# => Relation of all users
User.first	# => first user (ordered by id by default)
User.last	# => last user
User.find(3)	# => finds by primary key; raises if missing
User.find_by(email: "alice@codemash.org")	# => single record or nil (safe!)

User.first

User.second

User.first
User.second

User.third

User.first

User.second

User.third

User.fourth

User.first

User.second

User.third

User.fourth

User.fifth

User.first

User.second

User.third

User.fourth

User.fifth

User.forty_two

```
User.where(name: "alice")  
  .where("email LIKE ?", "%@codemash.org")  
  .order(:created_at)  
  .limit(5)
```

```
User.count # => integer count  
User.exists?(email: "bob@codemash.org") # => true/false
```

```
user = User.find_by(email: "alice@codemash.org")
```

```
# Classic attribute assignment + save
```

```
user.name = "alicia"
```

```
user.save
```

```
# One-liner mass assignment
```

```
user.update(name: "alicia", email: "alicia@codemash.org")
```

```
# triggers validations/callbacks
```

```
# Bang version for strict control
user.update!(name: "alicia")          # raises on validation failure

# Update many records at once (bypasses validations/callbacks)
User.where(name: "bob").update_all(name: "robert")
```

```
user = User.find(1)
```

```
user.destroy
```

```
# runs callbacks
```

```
(e.g., before_destroy), returns self
```

```
# Conditional destroy
```

```
User.where(name: "charlie").destroy_all # runs callbacks on each
```

```
# Raw delete (faster, skips callbacks & validations)
```

```
user.delete
```

```
User.where(name: "dana").delete_all
```

Active Record





Solid Cable

Solid Cache

Solid Queue



Solid Cable

Websockets Made Simple

★ Default Action Cable adapter in Rails 8

★ No



required!

What Does It Do?

- ★ Powers WebSockets (Action Cable) using your existing database
- ★ Stores broadcast messages in a database table
- ★ Uses fast polling (≈ 0.1 s intervals) → near real-time updates
- ★ Works great with SQLite, PostgreSQL, MySQL

Rock'em Socket'em Fight!

Feature	Classic (Redis)	Solid Cable (Rails 8+)
Extra service needed?	Yes (Redis)	No – just your DB
Deployment complexity	Higher	Much lower
Great for small/medium?	Overkill	Perfect fit
Performance	Excellent	Very good (tuned polling)
Default in new apps?	No	Yes!



Solid Cache

Caching Made Simple

- ★ Database-backed cache store – Default in Rails 8
- ★ Replaces Redis/Memcached as the default in production
- ★ Stores cache entries → in your existing database (PostgreSQL/MySQL/SQLite)
- ★ One less external service to run, monitor, or pay for

Wait ... Database?!

(Traditional wisdom: "Cache = super-fast in-memory (Redis)")

- ★ Modern SSDs/NVMe are *blazing fast*
- ★ Disk storage is much cheaper than RAM

Wait ... Database?!

(Traditional wisdom: "Cache = super-fast in-memory (Redis)")

- ★ Modern SSDs/NVMe are *blazing fast*
- ★ Disk storage is *much, much, much* cheaper than RAM

Wait ... Database?!

(Traditional wisdom: "Cache = super-fast in-memory (Redis)")

- ★ Modern SSDs/NVMe are *blazing fast*
- ★ Disk storage is *much, much, much* cheaper than RAM
- ★ You can cache way more data for longer (gigabytes instead of megabytes)

Sell Me This Cache ...

- ★ Zero extra infrastructure (no Redis instance, no config headaches)
- ★ Automatic cleanup (FIFO eviction when size limit reached)
- ★ Supports encryption
- ★ Plays nicely with multiple databases / sharding
- ★ Ships by default in new Rails 8+ apps

Use Cases

Solid Cache:

- Most small-medium apps
- Apps on platforms like Heroku, Render, Fly.io
- When you want to cache lots of fragments / pages

Use Cases

Use Redis/Memcached instead when:

- You need absolute top speed (sub-ms)
- Massive scale with very high write throughput



Solid Queue

Queue'n Made Simple

- ★ Default background job system in Rails 8
- ★ Handles async tasks: emails, image processing, reports, cleanups...
- ★ Replaces the need for Sidekiq/Resque
- ★ Zero new infrastructure

Queue'n Made Simple

- ★ Supports delays, priorities, retries (via Active Job)
- ★ Concurrency controls & queue pausing
- ★ High throughput (thousands of jobs/minute on modern DBs)
- ★ Mission Control dashboard (extra gem) → beautiful web UI for monitoring

Ok, Boomer ...

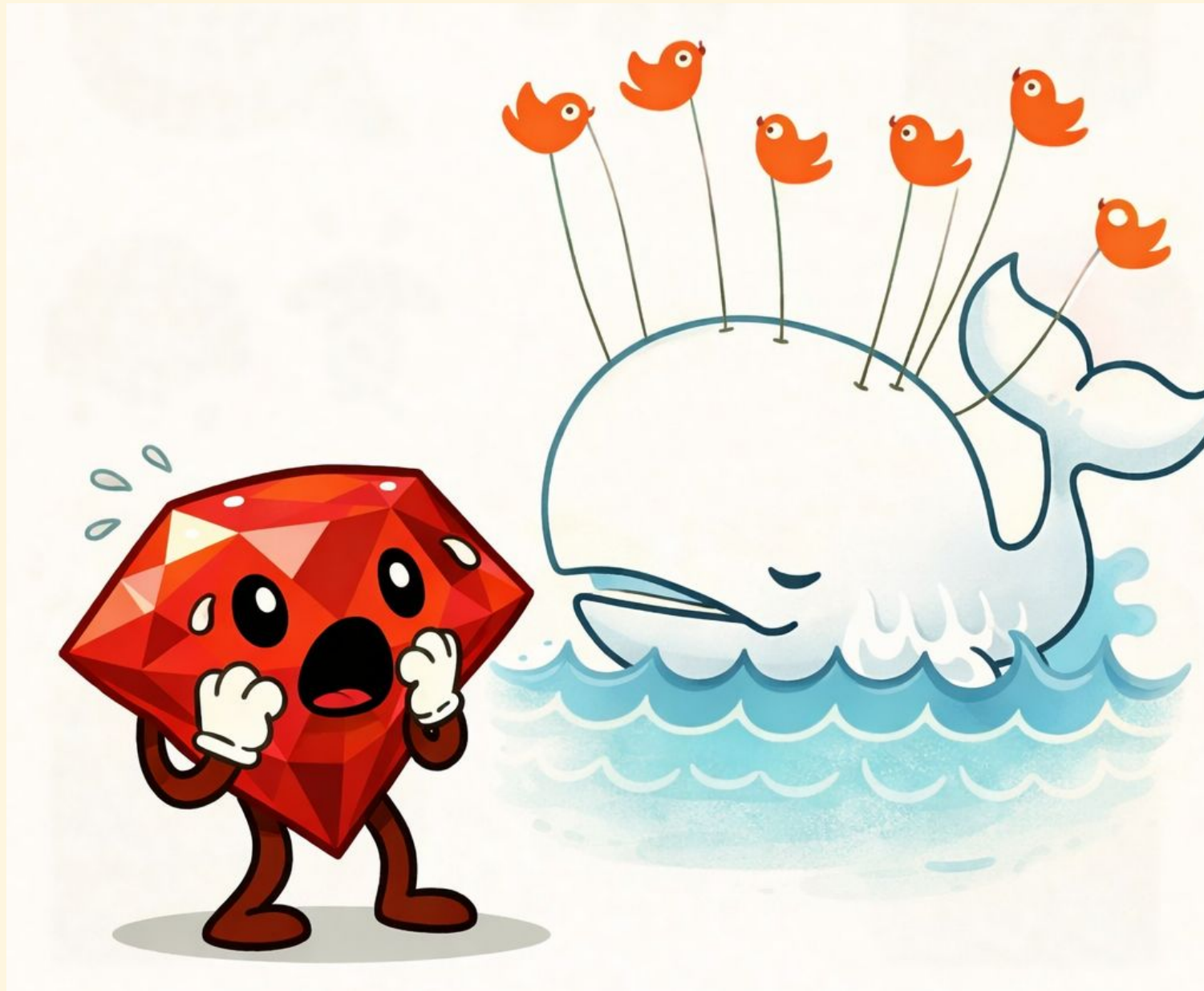
Old Way:

App → Sidekiq/Resque → Redis → Workers

New Way:

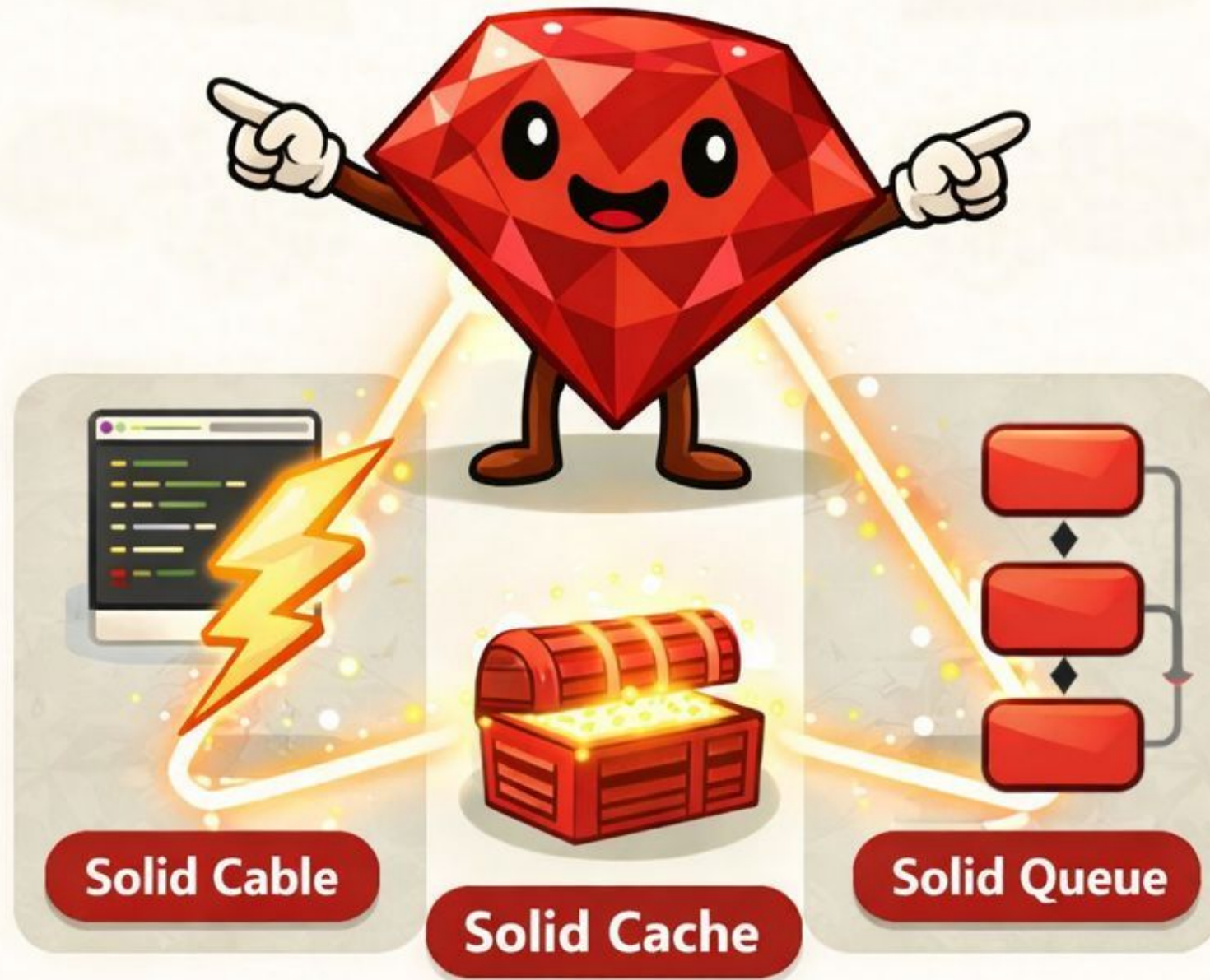
App → Your Existing Database! → Workers

Oh, Noes ...



Solid Trifecta

Ruby on Rails 8



"No one needs to become an operations expert to build a great Rails app."

- You can build fast, scalable, full-stack apps with almost zero extra moving parts
- More time coding, less time ops
- "One Person Framework"



Kamal
2.0

Deployin' Like It's 1999

- ★ Default with Ruby on Rails 8 – No extra gem install needed!
- ★ One command → production-ready server: kamal setup
- ★ Turns a plain Linux box (DigitalOcean, Hetzner, AWS EC2, your own hardware) into a full app server

Deployin' Like It's 1999

- ★ Zero-downtime deploys + rolling restarts built-in
- ★ No Nginx, no Traefik gymnastics → your Rails container faces the internet directly
- ★ "Deploy anywhere" — escape PaaS lock-in and crazy bills

Deployin' Like It's 1999

- ★ Kamal Proxy — custom, Rails-optimized proxy (replaces Traefik)
 - Auto SSL with Let's Encrypt
 - HTTP/2, asset caching, compression, X-Sendfile acceleration
- ★ Thruster — makes your container internet-ready out of the box

Deployin' Like It's 1999

- ★ Multi-app support — run several Rails apps on one server easily
- ★ Better secrets handling — native 1Password, Bitwarden, etc. integration

Rails Doctrine Ahoy!

- ★ Convention over configuration — sensible defaults, minimal setup
- ★ The joy of shipping — focus on building, not ops hell
- ★ Full-stack ownership — own your deployment like you own your code

Rails Doctrine Ahoy!

- ★ No PaaS middleman → true independence and cost control → Kamal 2 makes "deploying Rails" feel as delightful as writing Rails



Kamal
2.0



Propshaft

Ye 'Ol Asset Pipeline

- ★ Propshaft is the modern asset pipeline in Rails 8
- ★ Replaces Sprockets (the old default since Rails 3.1)

Ye 'Ol Asset Pipeline

- ★ Modern browsers changed the game:
 - HTTP/2 → many parallel requests are cheap (no need to bundle everything)
 - Much faster bandwidth → minification & heavy compression matter less



Propshaft



*Well,
Ackchyuually ...*

.../Development ➤ rails new

Usage:

rails new APP_PATH [options]

Options:

-G,	[--skip-git]	# Skip git init, .gitignore and .gitattributes
	[--skip-docker]	# Skip Dockerfile, .dockerignore ...
	[--skip-keeps]	# Skip source control .keep files
-M,	[--skip-action-mailer]	# Skip Action Mailer files
	[--skip-action-mailbox]	# Skip Action Mailbox gem
	[--skip-action-text]	# Skip Action Text gem
-O,	[--skip-active-record]	# Skip Active Record files
	[--skip-active-job]	# Skip Active Job
	[--skip-active-storage]	# Skip Active Storage files
-C,	[--skip-action-cable]	# Skip Action Cable files
-A,	[--skip-asset-pipeline]	# Skip the asset pipeline setup
-J, --skip-js,	[--skip-javascript]	# Skip JavaScript files
	[--skip-hotwire]	# Skip Hotwire integration
	[--skip-jbuilder]	# Skip jbuilder gem
-T,	[--skip-test]	# Skip test files
	[--skip-system-test]	# Skip system test files
	[--skip-bootsnap]	# Skip bootsnap gem
	[--skip-dev-gems]	# Skip development gems (e.g., web-console)
	[--skip-thruster]	# Skip Thruster setup
	[--skip-rubocop]	# Skip RuboCop setup
	[--skip-brakeman]	# Skip brakeman setup
	[--skip-bundler-audit]	# Skip bundler-audit setup
	[--skip-ci]	# Skip GitHub CI files
	[--skip-kamal]	# Skip Kamal setup
	[--skip-solid]	# Skip Solid Cache, Queue, and Cable setup

Full-Stack
Flexing



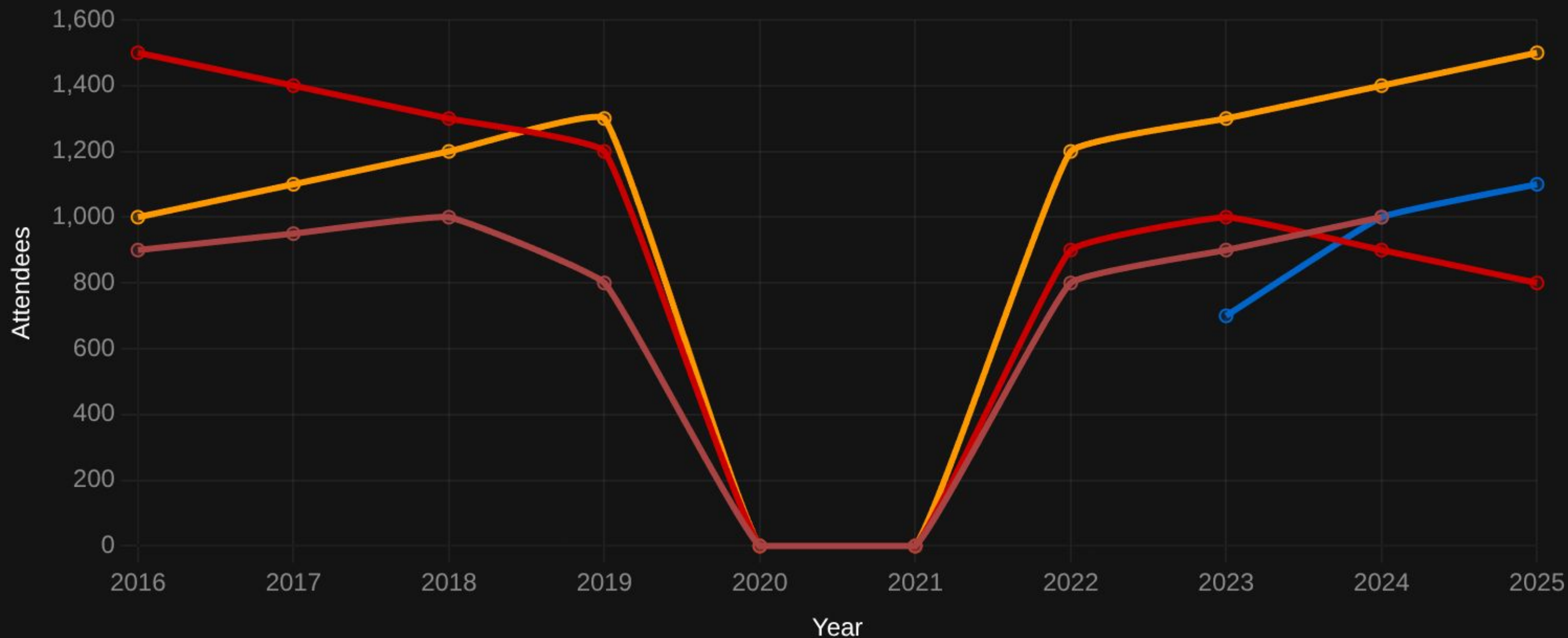
Community

<https://rubyonrails.org/community>

<https://www.ruby-lang.org/en/community>



Conference Attendance (2016–2025) — Sold-Out Years Highlighted!



- RubyConf (~Sold Out in many late-2010s years, incl. 2019)
- RailsConf (Sold Out 2025 final edition)
- RubyKaigi (Consistently Sold Out almost every year)
- RailsWorld (Sold Out EVERY year - 2023/24/25 in minutes!)

Ruby Friends

Meet someone new. Start a real conversation. Reconnect later.

Because the best part of a conference is who you meet.

Create my profile



<https://rubyfriends.app>



Break the ice without the awkwardness.



Turn the hallway track into real conversations.



Reconnect and make life-long Ruby friends.

Ruby Friends

<https://rubyfriends.app>

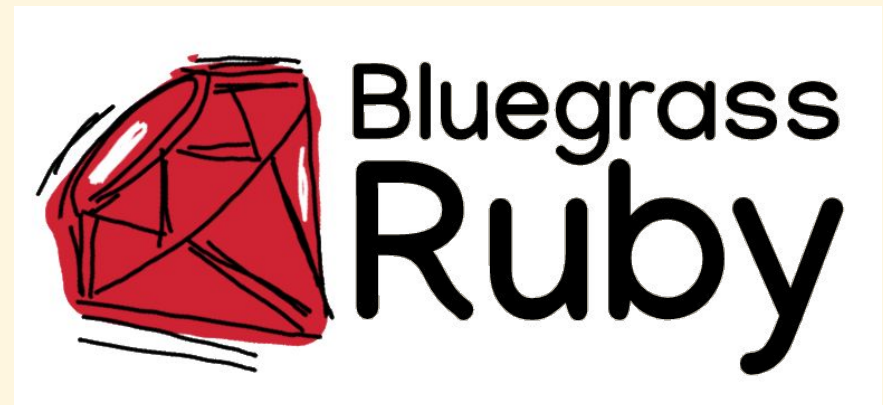


<https://rubyfriends.app/profiles/BMBY>

RubyEvents.org



indy.rb The Indianapolis Ruby Brigade





★ ***Columbus Ruby On Rails
Mind Meld***

★ ***Detroit.rb***

★ ***Ann Arbor Ruby Group***

★ ***DC Ruby Users Group
(DCRUG)***

★ ***Pittsburgh Ruby Group***

Full-Stack
Flexing





Let's

Talk

AI

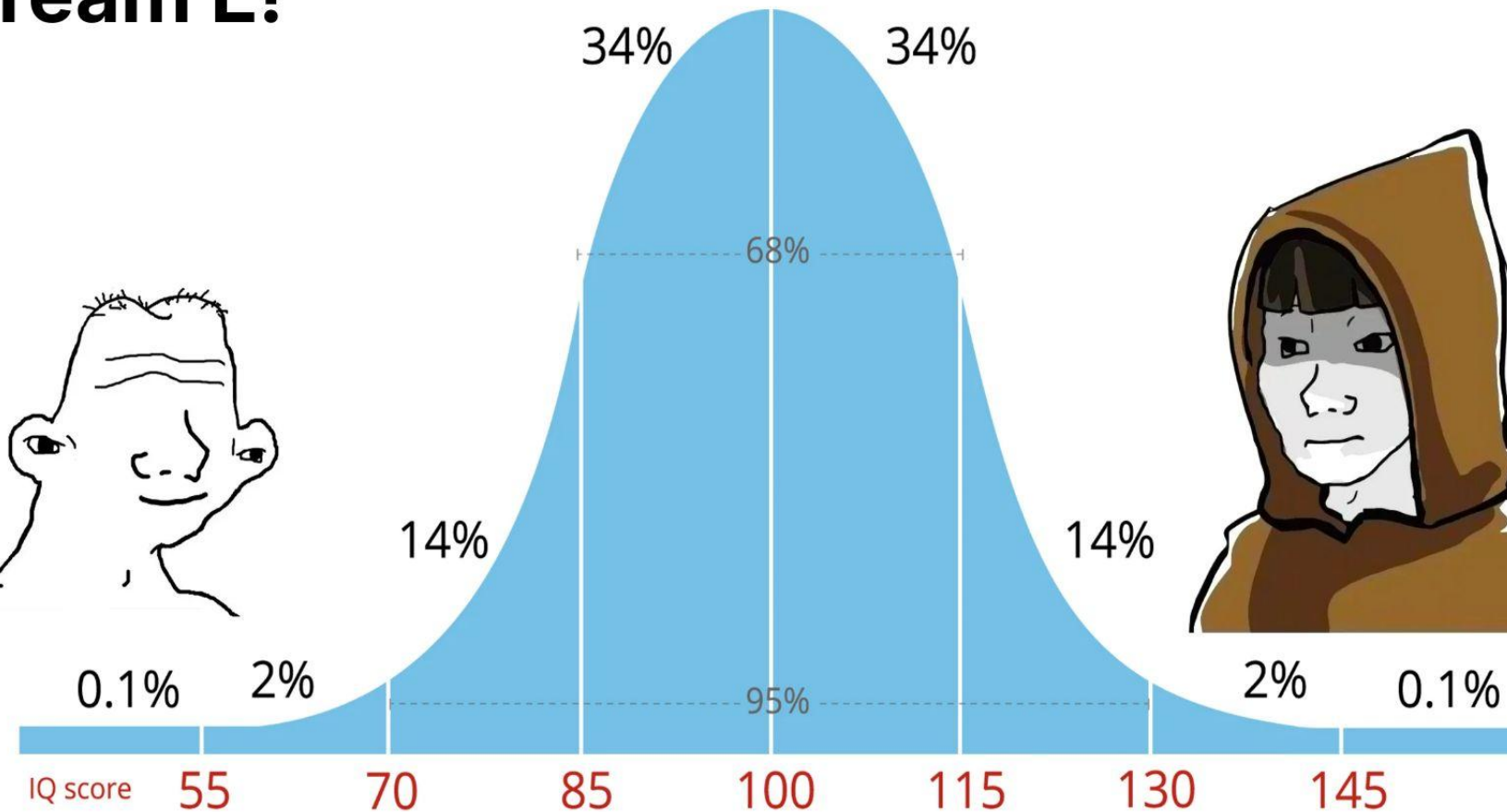
Team S!



Team F!



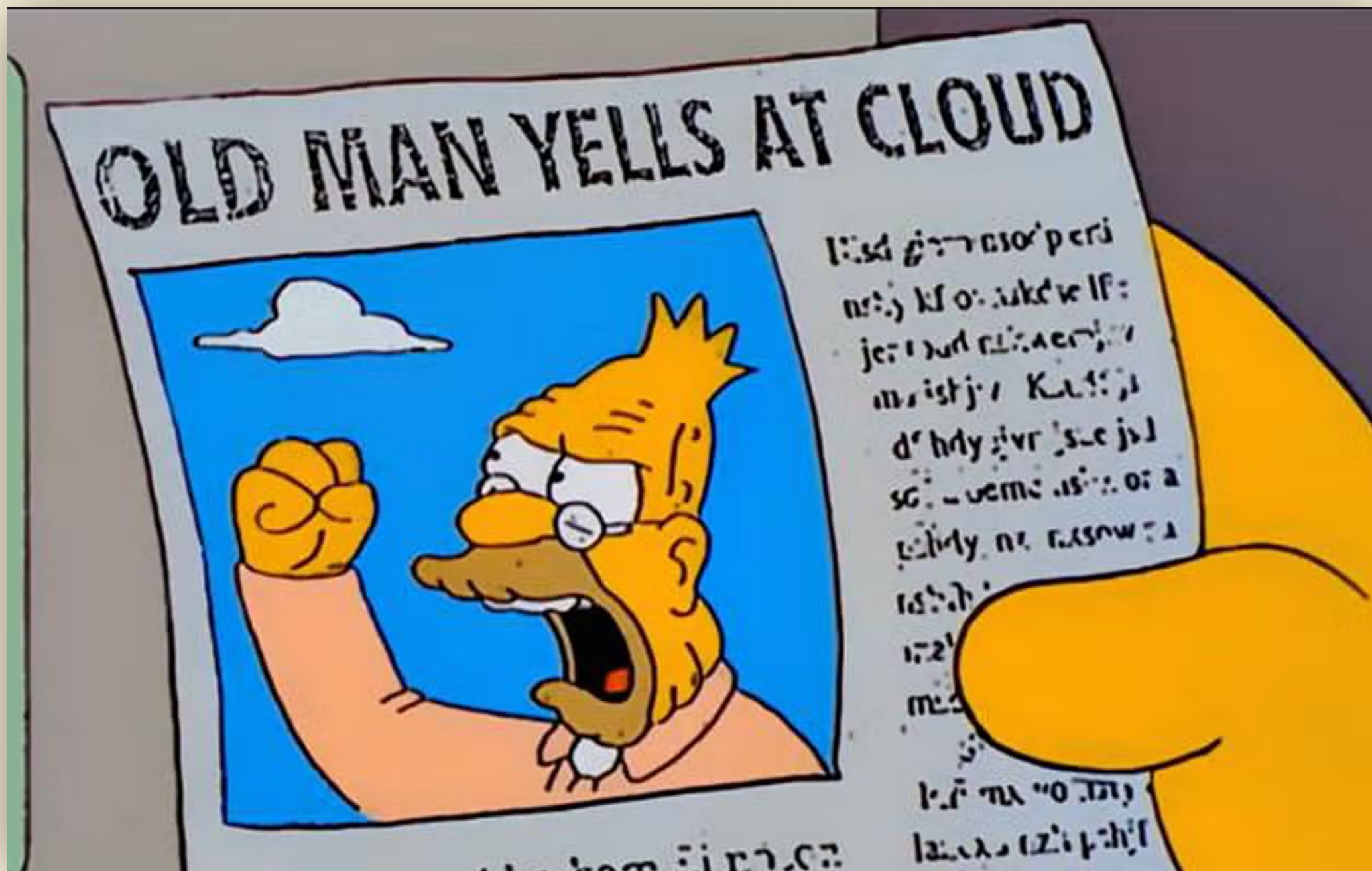
Team E!



Team Enthusiasm!



Team Skeptical!



Team Mouse Farts!



Ruby is ready for AI

Ruby is ready for AI

→ Readability

Ruby is ready for AI

→ Readability

→ Expressiveness

Ruby is ready for AI

→ Readability

→ Expressiveness

→ Developer Happiness

<https://try.ruby-lang.org>

Got 30 minutes? Give Ruby a shot right now!

Ruby is a programming language from Japan which is revolutionizing software development.

The beauty of Ruby is found in its balance between simplicity and power.

You can type some Ruby code in the editor and use these buttons to navigate:

- **Run** → Executes the code in the editor
- **Copy** → Copies the example code to the editor
- **Next** → Allows you to go to the next lesson
- **Back** → Allows you to return to the previous lesson



EDITOR

```
3.times do  
  print 'Welcome '  
end
```

[Copy](#)[Run](#)

OUTPUT

Welcome Welcome Welcome

[Back](#)[Got 30 minutes? Give Ruby a shot right now!](#)[Next](#)

<https://guides.rubyonrails.org>

(<https://www.ruby-lang.org/en/documentation/installation>)



Getting Started with Rails

This guide covers getting up and running with Ruby on Rails.

After reading this guide, you will know:

- How to install Rails, create a new Rails application, and connect your application to a database.
- The general layout of a Rails application.
- The basic principles of MVC (Model, View, Controller) and RESTful design.
- How to quickly generate the starting pieces of a Rails application.
- How to deploy your app to production using Kamal.

1. Introduction

Welcome to Ruby on Rails! In this guide, we'll walk through the core concepts of building web applications with Rails. You don't need any experience with Rails to follow along with this guide.

Rails is a web framework built for the Ruby programming language. Rails takes advantage of many features of Ruby so we **strongly** recommend learning the basics of Ruby so that you understand some of the basic terms and vocabulary you will see in this tutorial.

- [Official Ruby Programming Language website](#)
- [List of Free Programming Books](#)

2. Rails Philosophy

Rails is a web application development framework written in the Ruby programming language. It is designed to make programming web applications easier by making assumptions about what every developer needs to get started. It allows you to write less code while accomplishing more than many other languages and frameworks. Experienced Rails developers also report that it makes web application development more fun.

Chapters

■ 1. Introduction

2. Rails Philosophy

3. Creating a New Rails App

- Prerequisites
- Creating Your First Rails App
- Directory Structure
- Model-View-Controller Basics

4. Hello, Rails!

- Autoloading in Development

5. Creating a Database Model

- Database Migrations
- Running Migrations

6. Rails Console

7. Active Record Model Basics

- Creating Records
- Querying Records
- Filtering & Ordering Records
- Finding Records
- Updating Records
- Deleting Records



Full-Stack

Flexing

with

Ruby on Rails

